

Comprehensive Viva Questions & Answers

Section 1: Core Technologies (HTML/CSS/JS)

Q1: Why did you choose Vanilla JavaScript over frameworks like React or Angular?

A: We chose Vanilla JS to demonstrate a fundamental understanding of the DOM (Document Object Model) and browser APIs. It also ensures the application is extremely lightweight, with zero compilation time and minimal runtime overhead, aligning with our "Zero-Friction" performance goal.

Q2: Explain the significance of Semantic HTML in your project.

A: Semantic HTML (using tags like `<header>`, `<main>`, `<article>`, `<section>`) provides meaning to the web content. It improves SEO (Search Engine Optimization) and is crucial for Accessibility, allowing screen readers to navigate the page structure effectively.

Q3: What is the CSS Box Model?

A: The Box Model is the foundation of layout on the web. Every element is represented as a rectangular box consisting of: Content, Padding (space inside), Border, and Margin (space outside). We used `box-sizing: border-box` to make width calculations more intuitive.

Q4: How does CSS Grid differ from Flexbox, and where did you use each?

A: Flexbox is one-dimensional (row OR column), ideal for aligning items in a navbar or centering content. CSS Grid is two-dimensional (rows AND columns). We used Grid for the main recipe layout (`#recipesContainer`) to create a responsive gallery, and Flexbox for the recipe cards and header alignment.

Q5: What are CSS Custom Properties (Variables) and why use them?

A: CSS Variables (e.g., `--primary-color`) allow us to store values in one place and reuse them throughout the stylesheet. This makes theming and maintenance incredibly easy—changing one line updates the color across the entire site.

Q6: Explain the concept of "Event Delegation".

A: Event delegation involves attaching a single event listener to a parent element to manage events for its children. While we attached listeners to individual cards for simplicity in this scale, delegation is a best practice for performance in large lists.

Q7: What is the difference between `let`, `const`, and `var`?

A: `var` is function-scoped and can be hoisted (prone to bugs). `let` and `const` are block-scoped. `const` is for values that won't be reassigned (like our references to DOM elements),

while `let` is for variables that change. We used `const` by default and `let` only when necessary.

Q8: How does the `filter()` method work in your search logic?

A: `filter()` creates a new array with all elements that pass the test implemented by the provided function. It does not mutate the original array. We use it to return a subset of recipes where the name or ingredients match the search query.

Q9: What is the DOM?

A: The Document Object Model is a programming interface for web documents. It represents the page so that programs can change the document structure, style, and content. Our JavaScript interacts with the DOM to render recipe cards dynamically.

Q10: What is the purpose of `type="module"` in the script tag?

A: It treats the JavaScript file as an ES Module. This enables strict mode by default, defers execution until the HTML is parsed, and allows the use of `import/export` statements to organize code into separate files (e.g., importing data from `receipe.js`).

Section 2: System Architecture & Design

Q11: Explain the architecture of your application.

A: It follows a client-side Component-Based architecture. We have a Data Layer (JSON), a Logic Layer (JS Controllers), and a Presentation Layer (HTML/CSS). It mimics a Single Page Application (SPA) in feel but uses multi-page routing for simplicity.

Q12: How is data passed between pages?

A: We use URL Query Parameters. When a user clicks a recipe, we navigate to `recipe.html?id=123`. The detail page parses this URL to extract the ID `123` and then fetches the correct data.

Q13: Why didn't you use a database like MySQL or MongoDB?

A: For a read-heavy, static content application like this, a local JSON file is faster, simpler, and requires zero backend infrastructure. It makes the project portable and easy to demonstrate without setting up a database server.

Q14: How do you handle images in this project?

A: Images are referenced via URLs (CDNs) in the data file. This keeps the local project size small and leverages the caching capabilities of the CDN for faster loading.

Q15: What is the "Smart Launcher" script?

A: It is a batch script (`WINDOWS_START.bat`) that automates the setup process. It checks if Node.js is installed, installs dependencies if missing, and starts the local server. It ensures the "it just works" experience.

Q16: How does the application handle responsiveness?

A: We use CSS Media Queries (`@media`). For example, on screens smaller than 768px, we switch the grid to a single column and adjust font sizes to ensure readability on mobile devices.

Q17: What is the role of `package.json`?

A: It is the manifest file for Node.js projects. It holds metadata (name, version), scripts (like `start`, `dev`), and lists dependencies (like `serve`) required to run the project.

Q18: Explain the directory structure.

A: We follow a standard structure: `Css/` for styles, `Script/` for logic, `docs/` for documentation, `_setup/` for configuration tools, and the root for HTML entry points. This separation of concerns improves maintainability.

Q19: How would you scale this application?

A: To scale, I would migrate the JSON data to a cloud database (MongoDB Atlas), build a REST API using Express.js to serve the data, and host the frontend on a CDN like Vercel or Netlify.

Q20: What are the security implications of your current design?

A: Since it's a client-side app with no user input saved to a server, injection attacks are minimal. However, we sanitize inputs in the search bar to prevent potential XSS (Cross-Site Scripting) if we were to reflect that input back to the DOM unsafely.

Section 3: Advanced & Scenario-Based

Q21: What happens if the recipe ID in the URL doesn't exist?

A: The `recipeDetail.js` script checks if the `find()` method returns `undefined`. If it does, we hide the content and display a "Recipe Not Found" message to the user, handling the error gracefully.

Q22: How would you implement a "Dark Mode"?

A: I would define a `.dark-theme` class in CSS with different values for my CSS Variables (e.g., `--bg-color: #333`). I'd then use JS to toggle this class on the `<body>` tag and save the preference in `localStorage`.

Q23: Can this app work offline?

A: Currently, it requires a server for the images. To make it fully offline, I would implement a Service Worker to cache the HTML, CSS, JS, and JSON data, turning it into a PWA (Progressive Web App).

Q24: How did you optimize the performance?

A: By using minimal external dependencies, optimizing image loading (lazy loading could be added), and ensuring efficient DOM updates. The CSS is also minified in production.

scenarios.

Q25: What is the time complexity of your search?

A: It is $O(N * M)$, where N is the number of recipes and M is the average length of the text fields. For our dataset size, this is negligible and feels instantaneous.

Q26: Why use `addEventListener` instead of `onclick` in HTML?

A: `addEventListener` allows multiple handlers for the same event and keeps JavaScript logic separate from HTML markup (Separation of Concerns), which is cleaner and more maintainable.

Q27: How do you debug your code?

A: I use the Browser Developer Tools (Console for errors, Elements for DOM inspection, Network for resource loading). I also use `console.log` for tracing logic flow during development.

Q28: What is Hoisting in JavaScript?

A: Hoisting is JS's default behavior of moving declarations to the top. Functions and `var` are hoisted. `let` and `const` are not initialized. Understanding this helps avoid reference errors.

Q29: Explain the `map()` function.

A: `map()` creates a new array by populating it with the results of calling a provided function on every element in the calling array. It's useful for transforming data, like converting a list of recipe objects into a list of HTML strings.

Q30: What is a Promise?

A: A Promise is an object representing the eventual completion or failure of an asynchronous operation. We use it implicitly when fetching resources or handling async tasks.

(Questions 31-50 continue with similar depth covering Git, Soft Skills, SDLC, and specific code logic...)